Information Retrieval

Programming Assignment I

Group Name: Information Retrieval Group Project

Choudhary Asmi Aashay – 23CS30061
G. Sri Mahitha Pranavi – 23CS30021
Sanjana Kothi – 23CS10063

GitHub Repository:
https://github.com/Pranaviee/IR_Assgn1

1 Preprocessing

Before building the index, we first process the raw Cranfield collection. The collection contains 1400 documents and is stored in the file `cran.all.1400`. Each document contains several tagged sections. The `.I` tag gives the document identifier, while `.T` and `.W` contain the title and abstract. The collection also contains author and other information, but we do not use those fields.

We read the collection sequentially and keep track of the current section of each document. Text is collected only while we are inside a `.T` or `.W` section. When a new `.I` tag is reached, the title and abstract collected for the previous document are joined together and passed through the preprocessing steps.

1.1 Tokenisation

The first step is tokenisation. We use a regular expression to extract alphabetic sequences from the document text. This separates the text into individual word tokens and removes punctuation, numbers and other non-alphabetic characters.

For example, punctuation around a word does not become part of the token. The output of this step is simply a list of word strings.

1.2 Normalisation

Next, every token is converted to lower case. This makes words with different capitalisation equivalent. For example, `Flow`, `FLOW`, and `flow` are all treated as the same term.

Normalisation is done before stopword removal so that tokens can be compared directly with the lower-case words stored in the stopword list.

1.3 Stopword removal

We load the stopwords from `stopwords.txt` into a Python set. Using a set allows us to test whether a token is a stopword efficiently.

Each token is checked against this set, and tokens present in the stopword list are removed. The remaining tokens are passed to the next stage. Removing these common words reduces the number of terms that need to be stored in the index.

1.4 Stemming

Finally, we apply the Porter stemmer to every remaining token. Different forms of related words are therefore reduced to a common stem. For example, the index may store a stem instead of keeping several grammatical forms as different terms.

The same Porter stemmer is later used while processing query words. This is important because the terms used to search the index must be in the same form as the terms that were stored during preprocessing.

1.5 Writing the processed collection

After all four steps are completed, the processed tokens are written to a new file named

`Information_Retrieval_Group_Project_processed.all`

The document identifier is preserved using the `.I` tag, followed by a `.S` tag containing the processed tokens.

The output format is:

```
.I 1
.S
processed token1 processed token2 processed token3
.I 2
.S
processed token1 processed token2 processed token3
```

This processed collection is used as the input to the indexing stage. The separation between preprocessing and indexing also allows the processed file to be checked independently before constructing the inverted index.

2 Indexing

The indexing stage takes the preprocessed collection as input. Each document in this file begins with an `.I` line containing its document identifier, followed by an `.S` line and the processed tokens. The aim is to build an inverted index that maps every distinct token to the documents in which it occurs. The input file is named

Information_Retrieval_Group_Project_processed.all

2.1 Reading the processed collection

The file is read sequentially rather than loading the complete collection into memory. While reading a document, its tokens are stored in a set. This removes repeated occurrences of the same token within that document, since a Boolean index only needs to record whether a term is present.

When the next `.I` tag is encountered, the current document is added to the postings list of every term in its set. The final document is added after the end of the file is reached.

The parser also checks the structure of the input. It reports an error for a malformed document identifier, a repeated identifier, a missing or repeated `.S` tag, an unknown tag, or tokens occurring outside an `.S` section. These checks help prevent a damaged processed file from producing an incorrect index.

2.2 Building the postings lists

The index is represented by a dictionary whose key is a term and whose value is its postings list. Document identifiers are appended as the documents are read.

The processed collection is arranged in increasing document id order, so the postings lists are produced in ascending order.

A set of document identifiers is maintained separately to detect duplicates and to obtain the maximum document identifier. Before writing the file, the vocabulary is sorted lexicographically.

The first line of the index contains the vocabulary size and the maximum indexed document identifier. Every remaining line contains one term followed by its comma-separated postings list.

The format is therefore:

```
vocabulary_size, maximum_docid
term docid1,docid2,docid3
```

For example, a possible entry is:

```
experimental 1,7,9,21,27
```

The generated file is named

Information_Retrieval_Group_Project_cran.index

2.3 Correctness checks

The completed index is read again and checked against the in-memory postings.

The check verifies that the header is correct, terms are in strict lexicographic order, postings contain unique document identifiers in ascending order, every identifier is within the valid range, and the number of term lines matches the vocabulary size.

The index is accepted only if all these checks pass.

2.4 Complexity

Let T be the number of processed token occurrences, V the vocabulary size, and P the total number of term–document pairs.

Reading the collection and constructing the postings lists takes

$$O(T)$$

expected time.

Sorting the vocabulary takes

$$O(V \log V)$$

time, and writing the postings takes

$$O(P)$$

time.

The overall expected time complexity is therefore

$$\boxed{O(T + V \log V + P)}$$

while the index itself requires

$$\boxed{O(P)}$$

space.

3 Boolean Retrieval

In this part we use the index to answer queries. A Boolean query is an exact match query. We do not give scores to documents and we do not sort them by relevance. We only decide, for each document, whether it satisfies the query or not.

Our index stores, for every term, the sorted list of documents that contain that term. So answering a query means combining two sorted lists of numbers.

For an AND query we need the numbers present in both lists. For an OR query we need the numbers present in at least one list. Doing this combination quickly is the main problem in this section, and we implemented three different ways to do it.

3.1 Reading the index back from the file

We do not reuse the dictionary that was created during indexing. Instead, the function `load_idx` opens the index file again and reads it from the start.

It works in three steps.

First it reads the first line, which is the header, and throws it away, because the header only stores the vocabulary size and the largest document id.

Second, for every remaining line, it splits the line at the first space. The part before the space is the term. The part after the space is the postings list.

Third, it splits the postings part at every comma and converts each piece into an integer.

The final result is a dictionary called `idx_map`, where the key is a term and the value is a list of document ids.

We chose to read the file again for one main reason. If we read it again, then the search code depends only on the index file. It does not depend on any variable left in memory from the indexing part.

This means the search cells can be run alone. It also means that if the index file were written wrongly, we would find the problem here instead of searching an old copy in memory without knowing.

3.2 Processing the query words

A query word cannot be searched directly in the index. The index does not contain the word `flows`. It contains the stem `flow`. So the query word must be changed in the same way the document words were changed.

The function `process_term` does this. It applies the same four steps that were used during preprocessing, in the same order:

1. **Tokenisation.** The word is split into alphabetic tokens.
2. **Normalisation.** The token is converted to lower case.
3. **Stopword removal.** The token is removed if it is present in the stopword list. Our stopword file contains 358 words.
4. **Stemming.** The Porter stemmer reduces the token to its stem.

Using the same steps on both sides is very important. If the query used a different stemmer, or if it did not convert to lower case, then a word could be present in the collection but still return no documents.

Two results follow from this design, and both are worth writing down clearly:

- If a query word is a stopword, then step 3 deletes it. The word gives an empty postings list. So an AND query with that word returns no document at all.
- If a word survives all four steps but is not present in the vocabulary, then we also get an empty list. This happens because we look up the term using `idx_map.get(term,[])`, which returns an empty list instead of raising an error.

  So an AND query returns nothing, and an OR query returns the postings of the other word only. The program does not crash on an unknown word.

3.3 The types of queries we support

Our query parser is small and simple. The function `run_query` splits the query at spaces and then accepts only two forms.

The first form is a single word. In this case we process the word and return its postings list directly.

The second form has exactly three parts, written as

```
term1 OP term2
```

Here `OP` must be AND or OR. We convert the operator to upper case first, so `and` and `And` are also accepted.

Any other input raises an error. If the operator is not AND or OR, we report an unsupported operator. If the number of parts is not one or three, we report an invalid query format.

We did not implement the NOT operator. We also did not implement brackets or queries with more than two words, because the assignment only asks for one Boolean operator between two words.

The function `execute_query` is a small wrapper around this. It creates a Porter stemmer, runs the query, and writes the matching document ids into a file named

```
<query_id>.txt
```

with one id on each line. It also prints the number of matches so we can check the result quickly.

3.4 Sample queries

We tested the search with three queries. The results are shown in Table 1.

These numbers are what we expected. The two AND queries join a common word with a rare word, so they return only a few documents.

The OR query joins two very common words from aerodynamics, so it returns 513 documents, which is more than one third of the whole collection.

Query id	Query	Number of matches
q1	aerodynamic AND slipstream	5
q2	boundary OR layer	513
q3	experimental AND destalling	2

Table 1: Sample queries and the number of documents returned. The results are written to `q1.txt`, `q2.txt` and `q3.txt`.

3.5 Method 1: the two-pointer merge

This is the basic method. Both postings lists are already sorted, so we can move through both of them at the same time using one pointer for each list. Because of this, we never need to look at the same element twice.

For AND (`intersect_postings`).

We compare the two document ids that the pointers are currently pointing at. There are three cases.

- If the two ids are equal, the document contains both terms. We add it to the result and move both pointers forward by one.
- If the id in list A is smaller, we move the pointer of A forward by one.
- If the id in list B is smaller, we move the pointer of B forward by one.

Moving only the smaller one is safe. The other list is sorted and has already crossed that value, so that value can never appear in it again.

The loop stops as soon as any one list finishes, because after that no more common documents are possible.

For OR (`union_postings`).

The logic is almost the same, but here nothing is thrown away. We add the smaller of the two ids to the result and move that pointer forward.

If the two ids are equal, we add the id only once and move both pointers, which stops duplicates from entering the result.

When one list finishes, two small loops copy all the remaining ids from the other list into the result.

Time taken.

In every comparison at least one pointer moves forward. So each element is visited at most once, and the time is

$$\boxed{O(|A| + |B|)}$$

for both operators, where $|A|$ and $|B|$ are the sizes of the two lists.

This method is simple and hard to beat in the general case. Its weakness is that it treats both lists equally. It has to walk through the full long list even when the short list has only one element in it. The next two methods try to fix this weakness.

3.6 Method 2: binary search with a moving lower bound

When one list is much smaller than the other, that is when

$$|A| \ll |B|,$$

it is better to go through the small list only, and for each of its few ids ask whether that id is present in the big list.

A binary search answers this question in about

$$\log |B|$$

comparisons instead of walking through the list.

The function `intersect_postings_binary_optimized` does this. It first swaps the two lists if needed, so that `a_list` is always the smaller one.

Because of this swap, the caller does not have to worry about the order in which the lists are passed.

The range narrowing idea.

A simple version would search the whole big list every single time. This repeats a lot of work.

But the elements of the small list are given to the search in increasing order. So each new id must lie further to the right in the big list than the previous id did.

We use this fact. We keep a variable called `left` and pass it to `bisect.bisect_left` as the `lo` argument.

This makes every search start from the place where the last search ended, instead of starting from the beginning. The searching window becomes smaller and smaller as we move forward.

There is one detail here that is easy to get wrong.

When the id is found at position `pos`, we set

```
left=pos+1
```

because that position is now used and cannot match anything else.

But when the id is not found, we set

```
left=pos
```

and not `pos+1`.

The reason is that the element at position `pos` is bigger than the id we searched for. It was not a match this time, but it may still be a match for a later id from the small list. If we moved past it, we could miss a real match.

Time taken.

We do $|A|$ searches and each search costs at most

$$O(\log |B|).$$

So the total time is

$$\boxed{O(|A| \log |B|)}.$$

This is better than $O(|A| + |B|)$ when

$$|A| \log |B| < |A| + |B|,$$

which happens when the two lists differ a lot in size.

Using the same idea for OR.

The function `union_postings_binary` uses the same idea for union, but it does not fit as well.

First we copy the big list, and this copy becomes our result. Then, for each id of the small list, we binary search in the result. If the id is not present, we insert it at the correct position.

The search is fast, but the insertion is not. When Python inserts an element in the middle of a list, it has to shift all the elements after that position by one place. This costs

$$O(N)$$

in the worst case.

So this method is useful only when the small list is really small and almost no insertion happens. If the small list were large, this method would become slower than the simple merge.

3.7 Method 3: skip pointers

The third method keeps the linear scan, but allows it to jump forward.

In `intersect_postings_skips` we first compute a jump size for each list.

The jump size is

$$\left\lfloor \sqrt{L} \right\rfloor,$$

where L is the length of that list.

This value is the usual choice because it balances two opposite problems. A big jump size means fewer jumps, but each jump has a higher chance of going too far. A small jump size is safer, but then each jump saves very little.

How the jump works.

Suppose the two pointers do not match and the id in list A is bigger. Then the pointer of list B has to move forward.

Before moving it by one, we look ahead at the element

```
b_list[j+step_b]
```

- If that element is still smaller than or equal to `a_list[i]`, then no element between the current position and that position can be a match. So we jump directly by the full step.

  We repeat this as long as the condition is true.

- If that element is bigger, then jumping would go too far and we might skip a real match.

  In this case we move forward by one position only, exactly like the normal merge.

The same check is applied in the other direction when the pointer of list A has to move.

There is also a condition

```
step>1
```

which turns off jumping for very short lists, where a step of one would have no meaning.

No jump ever crosses a document that could have matched. So the output of this method is exactly the same as the output of the simple merge.

Time taken.

In the worst case no jump is taken at all, and the time is still

$$\boxed{O(|A| + |B|)}.$$

Skip pointers do not improve the order of the running time. They only reduce the constant factor, that is, the actual number of elements we look at.

Why skip pointers cannot be used for OR.

A union must return every document that appears in either list. So there is no element that we can safely jump over.

Jumping over an element would mean leaving it out of the answer, which would make the result wrong.

Skip pointers help only when some work can be thrown away, and a union throws away nothing. For this reason we use the simple two-pointer merge for OR, and we report the skip method as "not applicable".

3.8 Summary of the three methods

Table 2 compares the running times of the three methods.

Method	AND	OR
Two-pointer merge	$O(\|A\| + \|B\|)$	$O(\|A\| + \|B\|)$
Binary search	$O(\|A\| \log \|B\|)$	$O(\|A\|(\log \|B\| + \|B\|))$ worst case
Skip pointers	$O(\|A\| + \|B\|)$ worst case	Not applicable

Table 2: Time taken to combine two postings lists of sizes $|A|$ and $|B|$, where $|A| \leq |B|$.

The union entry of the binary method also includes the cost of shifting elements during insertion.

3.9 How we selected the test terms

For the experiment we needed two terms whose lists differ a lot in size. We did not want to choose them by hand, so the notebook selects them using a fixed rule.

First, we sort the whole vocabulary by the length of the postings list. The last term in this sorted order is the most frequent term of the collection.

This term is `flow`, and it appears in 730 documents.

Then we go through the sorted list from the shortest side and look for the first term that satisfies two conditions at the same time:

1. It must share at least one document with `flow`. Otherwise the intersection would be empty and the test would not be meaningful.
2. Its postings list must be shorter than one tenth of the postings list of `flow`. This makes sure the two sizes are really different.

The first term that passes both conditions is `abbrevi`, which appears in only one document.

The common document between `abbrevi` and `flow` is document number 122.

So our test case has a size ratio of

$$\boxed{1 : 730}.$$

This is close to the most extreme case that this collection can give. It is exactly the situation for which the two alternative methods were designed.

So the results below should be read as the best case for those methods, and not as the result of a normal query.

3.10 Checking correctness before measuring speed

A wrong answer is useless even if it is fast. So before measuring the time, we check that all three methods give the same output.

The notebook compares the results element by element. It confirms that the binary search intersection, the skip pointer intersection and the binary insertion union all give exactly the same lists as the two-pointer merge.

Only after these checks pass do we measure the running time.

3.11 Timing results

Each method was measured with the `timeit` module over 10,000 repetitions.

We used a large number of repetitions so that the clock resolution and small disturbances from the operating system do not affect the result.

Table 3 gives the total time of these 10,000 runs. The speedup is calculated against the two-pointer merge of the same operator.

Operation	**Method**	**Time (s)**	**Speedup**
AND	Two-pointer merge	0.07600	—
AND	Binary search	0.00256	29.7×
AND	Skip pointers	0.04266	1.78×
OR	Two-pointer merge	0.42768	—
OR	Binary insertion	0.00746	57.3×
OR	Skip pointers	Not applicable	—

Table 3: Total time for 10,000 runs on the postings lists of `abbrevi` (1 document) and `flow` (730 documents).

3.12 Explanation of the results

AND with binary search is about 30 times faster.

There are two separate reasons for this, and both matter.

The first reason is the algorithm. Our small list has only one element. So the binary search does about

$$\log_2 730 \approx 10$$

comparisons and stops.

The two-pointer merge, on the other hand, has to walk through the list of `flow` until it reaches document 122.

The second reason is the language. The `bisect` module is written in C, so the whole search runs below the Python interpreter.

The two-pointer merge runs its loop inside Python, so it pays the interpreter cost in every single iteration.

Therefore one part of this 30 times gap comes from the algorithm and the other part comes from the implementation.

AND with skip pointers is only about 1.78 times faster.

This gain looks small, and it is useful to understand why.

The jump size for `flow` is

$$\left\lfloor \sqrt{730} \right\rfloor = 27.$$

So we look at roughly one element out of every 27 in the part of the list that we must cross.

Even then we get only a 1.78× improvement.

The reason is that skipping does not remove the Python loop. It only makes each iteration cover more distance, while doing a little extra work in each iteration, namely one extra index calculation and one extra comparison for the lookahead.

So we reduce the number of iterations, but we do not remove the cost of running a loop in Python.

The binary search removes that loop completely, and this is why it wins by a much larger margin.

One more point is worth noting. The jump size for `abbrevi` is

$$\left\lfloor \sqrt{1} \right\rfloor = 1.$$

So the condition `step>1` turns off skipping for the small list, and the whole benefit comes from jumping inside the list of `flow`.

OR with binary insertion is about 57 times faster.

This is the biggest gap in our results, and here the reason is almost entirely about implementation and not about the algorithm.

Both methods produce the same 730 document ids. The difference is in how they build the output.

The two-pointer merge appends the ids one by one inside a Python loop.

The binary method calls

```
list(b_list)
```

which copies the whole list in one operation written in C, and after that it does only one search and at most one insertion for the single document of `abbrevi`.

So the same amount of work in theory takes very different time in practice.

3.13 Conclusions

1. For an AND query where a rare term is combined with a common term, binary search with a moving lower bound is clearly the best of the three methods. This case is also common in practice, because real queries often mix one rare word with one frequent word.

2. Skip pointers are not the right choice for this particular test case. Their real strength is in intersecting two lists of similar and medium size, where the repeated binary searches stop being worth their cost.

   Our $1 : 730$ pair is exactly the shape that favours binary search, which is why the skip method gives only $1.78\times$.

3. For an OR query, the simple two-pointer merge is the correct general answer, because every element has to be written to the output anyway.

   The only exception is when one list is very small. In that case copying the big list in one step and inserting the few extra ids is much faster.

4. The main lesson is that no single method is the best in all situations.

   A real system would compare $|A|$ and $|B|$ at query time and then choose the method: binary search when the two sizes are very different, and a normal or skip-based merge when the two sizes are close.